Rogue Tower

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

A suspicious cell tower has been detected in the network. Analyze the captured network traffic to identify the rogue tower, find the compromised device, and recover the exfiltrated flag.

💡 Hints

- Look for unauthorized test network broadcasts on UDP port 55000
- Find the device that connected to the rogue tower by checking HTTP User-Agent headers
- The encryption key is derived from the victim device's IMSI
- The exfiltrated data is split across multiple HTTP POST requests

---

🧰 Tools Used

- `file` – Identify PCAP file type
- **Wireshark** – GUI tool for analyzing network traffic
- `tshark` – Command-line Wireshark tool for packet analysis
- `cut` – Extract specific fields from output
- `xxd` – Convert hexadecimal to binary data
- `base64` – Decode Base64-encoded content
- `sort` / `uniq` – Organize and deduplicate output
- **CyberChef** – Online tool for XOR decryption and data transformation

---

🔍 Complete Solution

Step 1: Download and Prepare

Download the PCAP file named `rogue_tower.pcap` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type and general information:

```
file rogue_tower.pcap
```

Expected output:

```
rogue_tower.pcap: pcap capture file, microsecond ts (little-endian) - version 2.4 (Raw IPv4, capture length 65535)
```

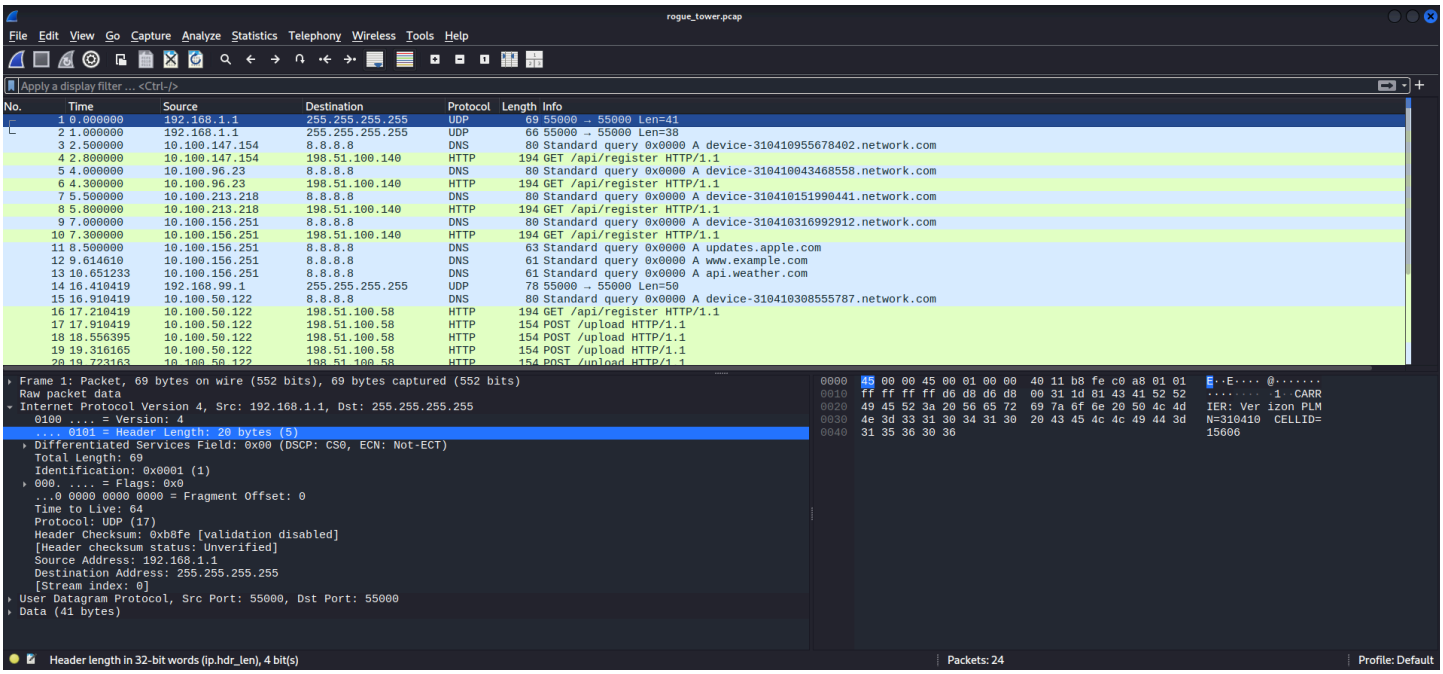
The file is a network packet capture (PCAP) that requires analysis using network forensics tools.

Step 3: Analyze with Wireshark (Visual Inspection)

Open the file in Wireshark to get a visual understanding of the traffic:

Upon inspection, observe:

- Repeated TCP packets with small payload lengths
- Most payloads contain Base64-like or hexadecimal characters
- Payload lengths are consistent: either 4 bytes or 12 bytes



> **Note**: Manually extracting each packet would be time-consuming, so we'll use `tshark` (command-line Wireshark) for automated extraction.

Step 4: Identify the Rogue Tower (UDP Port 55000)

The hint mentions **unauthorized test network broadcasts on UDP port 55000**. Let's examine these packets:

```
tshark -r rogue_tower.pcap -T fields -e frame.time -e data -Y "udp.port==55000"
```

Output:

```
2026-02-04T23:53:52.000000000+0200
434152524945523a20566572697a6f6e20504c4d4e3d333130343130204345 4c4c49443d3135363036
2026-02-04T23:53:53.000000000+0200
434152524945523a204154265420504c4d4e3d333130343130204345 4c4c49443d3135363037
2026-02-04T23:54:08.410419000+0200
554e415554484f52495a45442d544553542d4e4554574f524b20504c4d4e3d3030313031204345 4c4c49443932303538
```

The `data` field contains **hexadecimal strings**. Let's decode them:

```
tshark -r rogue_tower.pcap -T fields -e frame.time -e data -Y "udp.port==55000" | cut -f2 -d $'\t' | xxd -r -p
```

Output:

```
CARRIER: Verizon PLMN=310410 CELLID=15606CARRIER: AT&T PLMN=310410
CELLID=15607UNAUTHORIZED-TEST-NETWORK PLMN=00101 CELLID=92058
```

> **Key finding**: The **rogue tower** is the one broadcasting `UNAUTHORIZED-TEST-NETWORK` with `CELLID=92058` .

Step 5: Find the Compromised Device (HTTP User-Agent)

Now, let's examine the HTTP User-Agent headers to identify which device connected to the rogue tower (`CELLID=92058`):

```
tshark -r rogue_tower.pcap -T fields -e http.user_agent -Y "http.request.method==GET
|| http.request.method==POST" | uniq | sort
```

Output:

```
MobileDevice/1.0 (IMSI:310410043468558; CELL:15606)
MobileDevice/1.0 (IMSI:310410151990441; CELL:15606)
MobileDevice/1.0 (IMSI:310410308555787; CELL:92058)
MobileDevice/1.0 (IMSI:310410316992912; CELL:15606)
MobileDevice/1.0 (IMSI:310410955678402; CELL:15606)
```

> **Key finding**: The device with **IMSI:310410308555787** connected to `CELLID=92058` (the rogue tower). This is the **victim device**.

Step 6: Extract the Exfiltrated Data (HTTP POST Requests)

The hint states that exfiltrated data is split across multiple HTTP POST requests. Let's extract all POST data:

```
tshark -r rogue_tower.pcap -T fields -e frame.time -e http.file_data -Y
"http.request.method==POST" | cut -f2 -d $'\t' | xxd -r -p
```

Output:

```
QFFWWnZjfkxCCFJABmhbBFxUakEFQAtFb1xXVgEHAAQBRQ==
```

The data is actually a **single Base64 string** (split across multiple POST requests but reassembled by Wireshark). Let's decode it:

```
echo 'QFFWWnZjfkxCCFJABmhbBFxUakEFQAtFb1xXVgEHAAQBRQ==' | base64 -d
```

Output:

```
@QVZvc~LB□R@□h[□\TjA□@□Eo\WV□□
```

This appears to be **XOR-encoded** data (non-printable characters).

Step 7: Derive the XOR Key

The hint says: *"The encryption key is derived from the victim device's IMSI"*

The victim's IMSI is: `310410308555787`

Breaking it down:

- `310410` = PLMN (Mobile Network Code)
- `308555787` = MSIN (Subscriber ID)

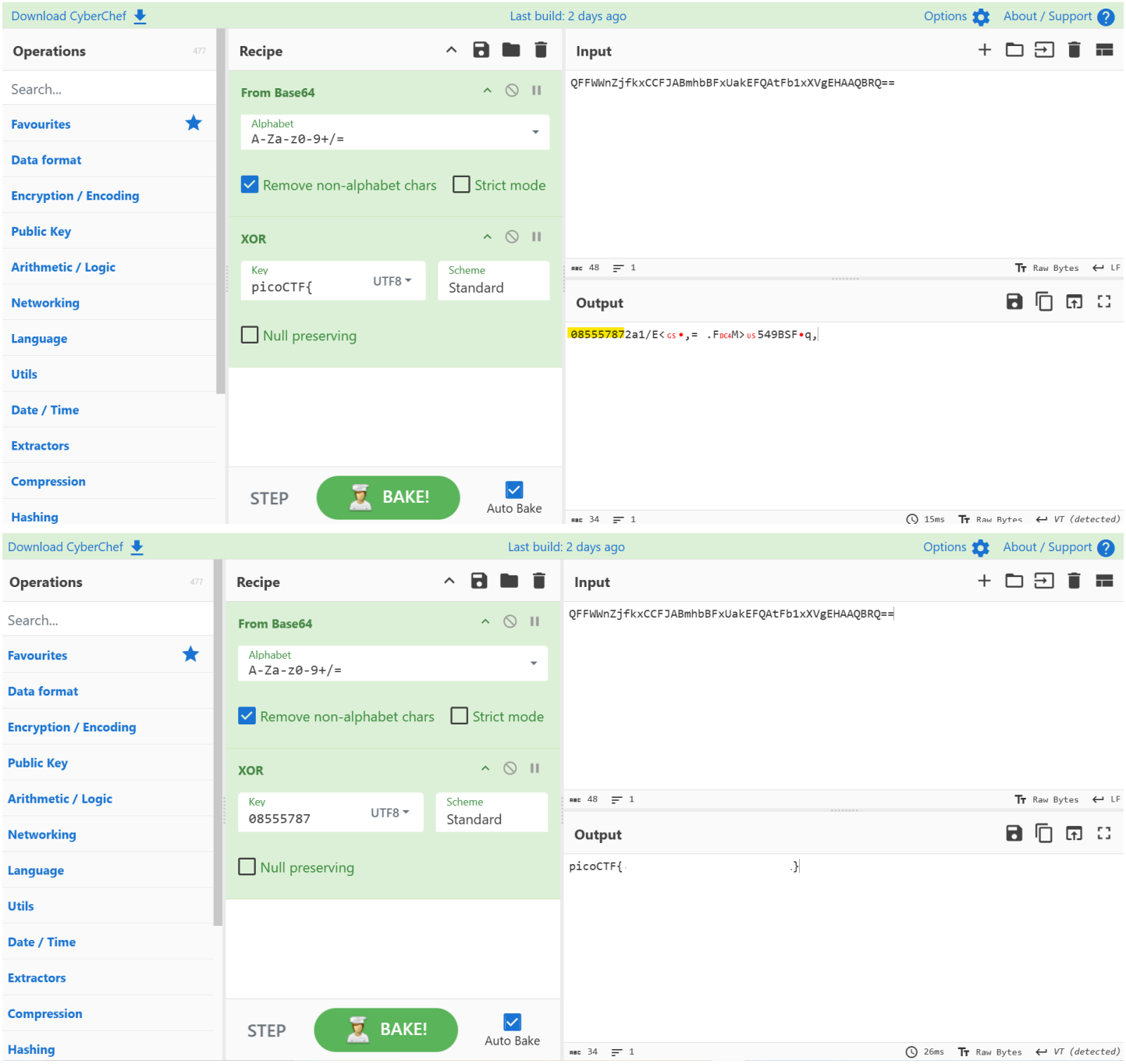
The hint suggests the key is derived from the IMSI. Often, the key is the **MSIN without the first digit** or a specific portion.

Let's try `08555787` (removing the first digit `3` from `308555787`).

Alternatively, we can use a **known plaintext attack** with the flag prefix `picoCTF{` to determine the XOR key.

Step 8: Decrypt with XOR (CyberChef)

Open CyberChef and configure:



✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, XOR decryption with the correct IMSI-derived key will reveal the complete flag.

🧠 Key Concepts

- **PCAP Analysis**: Network packet captures contain timestamped records of all traffic, useful for forensic investigations.
- **Rogue Tower Detection**: Unauthorized cell tower broadcasts can be identified by unusual PLMN/CELLID combinations (e.g., `PLMN=00101`).
- **UDP Port 55000**: Often used for cell tower broadcast messages (Carrier, PLMN, CELLID).
- **IMSI (International Mobile Subscriber Identity)**: A unique identifier for mobile subscribers, consisting of PLMN (MCC+MNC) + MSIN.
- **HTTP User-Agent**: Can reveal device identifiers like IMSI when devices register with cell towers.
- **XOR Encryption**: A simple symmetric cipher where the same key is used to encrypt and decrypt.
- **Known Plaintext Attack**: Using the known flag prefix `picoCTF{` to derive the XOR key.
- **tshark**: Command-line version of Wireshark, ideal for automated and scriptable packet analysis.
- **CyberChef**: A versatile web-based tool for decoding, decrypting, and transforming data.